CS3400 Class Activity (Submitted and Marked)

Instructions

- Late submissions will not be accepted.
- Mark allocation at the end of the document.
- Submit a compressed (zipped) folder named XXX_CS3400Activity, where XXX is your student number.
- The folder must contain exactly 4 .cpp files:
    - One file covering Questions 1, 2, and 3.
    - One file for Question 4.
    - One file for Question 5.
    - One file for Question 6.

1. Completing the Skeleton
   Based on the linked list skeleton provided earlier, complete all the missing code segments as indicated by the embedded comments.
   a. Ensure that your implementation compiles and runs correctly.
   b. Include brief inline comments explaining how each missing piece fits into the full linked list functionality.

2. Encapsulation and Node Design
   From our class discussion, we emphasized that encapsulation is a cornerstone of Object-Oriented Programming (OOP) and must always be respected.
   a. Redesign and improve the Node class by making all member variables private.
   b. Provide appropriate getter and setter methods.
   c. Modify the linked list class accordingly to work with the encapsulated Node.
   d. Add comments highlighting what you changed and why, for clarity.

3. Destructor Implementation
   Memory leaks are a common issue when working with dynamic memory allocation in C++.
   a. Implement a destructor for your linked list class to ensure all dynamically allocated memory is properly freed when the list object is destroyed.
   b. Demonstrate the destructor in action (e.g., by creating and deleting a list in main()).

4. GPS Coordinates – Application of Linked List
   Extend your linked list to represent GPS Coordinates / Map Routes (Use a helper distance function (like Haversine) for realistic geolocation distances.)
   a. Each Node should contain:
      i. Latitude
      ii. Longitude
      iii. Timestamp
   b. Implement the following operations:
      i. Insert Waypoints into the path (maintaining order by timestamp).
      ii. Calculate Total Distance Traveled using the coordinates.
      iii. Search for the Nearest Point to a given location (hint: compute Euclidean or Haversine distance).
      iv. Delete Duplicate Waypoints if the same coordinates appear consecutively.
      v. Reverse the Route (linked list reversal).

5. Bank Transactions – Menu-Driven Doubly Linked List
   Implement a menu-driven program using a doubly linked list to manage bank transactions.
   a. Each Node should contain:
      i. TransactionID
      ii. Amount
      iii. Date
      iv. Type (Deposit/Withdrawal)
   b. Implement the following operations:
      i. Sum Deposits vs Withdrawals (calculate account balance).
      ii. Detect Overdrafts (when withdrawals exceed deposits).
      iii. Find the Largest Transaction in the history.
      iv. Delete Transactions Older than 1 Year.
      v. Generate a Mini-Statement (show last 5 transactions).
   c. Additions
      i. Implement input validation, e.g., rejecting negative amounts or invalid dates.
      ii. Enhance your program by implementing file persistence: save the transaction list to a file before the program exits, and reload the data from the

file when the program starts. This ensures that transaction history is preserved across multiple runs.

6. Bonus Marks
   a. Templates: Generalize the linked list implementation using C++ templates so it can store any data type.
   b. Exception Handling: Add error checking and throw exceptions for invalid operations (like deleting from an empty list).
   c. At the start of your template implementation, include a block comment that explains the trade-offs between singly linked lists, doubly linked lists, and circular linked lists. Relate your explanation to the real-world problems implemented above, highlighting which list type is most suitable for each scenario and why.

Mark Allocation

Question	Task	Marks
Q1	Complete skeleton code	10
Q2	Encapsulation + redesign Node class	10
Q3	Destructor implementation	10
Q4	GPS Route operations	25
Q5	Bank Transactions operations	30
Q6	Additions (validation + persistence)	5
Bonus	Templates, exceptions, trade-offs	+15